

RAPPORT TECHNIQUE

Application Bancaire

Spring Boot + React

BankApp — Système de gestion bancaire full-stack

Version : 1.0

Date : 5 mai 2026

1. Introduction

BankApp est une application web de gestion bancaire développée en architecture full-stack. Elle repose sur un backend RESTful construit avec Spring Boot 3 et un frontend réactif développé avec React. L'objectif du projet est de fournir une interface moderne permettant la gestion des clients, des comptes bancaires, la consultation des transactions et l'exécution de virements entre comptes.

Ce rapport présente l'architecture globale du projet, les choix technologiques, la structure de la base de données, les entités JPA, les endpoints REST exposés, ainsi qu'une analyse du code source.

2. Architecture Globale

Le projet adopte une architecture en couches (layered architecture) clairement séparée entre le backend et le frontend :

Couche	Description
Frontend	Application React (Vite) communiquant via Axios avec l'API REST
Controller	Couche REST : réception et dispatch des requêtes HTTP
Service	Couche métier : logique applicative et validation des règles
Repository	Accès aux données via Spring Data JPA (interfaces)
Model / BDD	Entités JPA persistées dans une base H2 en mémoire

2.1 Structure du projet

```
spring-bank-project/
├── backend/
│   ├── src/main/java/com/bankapp/
│   │   ├── config/           # Configuration CORS
│   │   ├── controller/      # Contrôleurs REST
│   │   ├── dto/             # Objets de transfert (LoginRequest, etc.)
│   │   ├── model/           # Entités JPA
│   │   ├── repository/      # Interfaces Spring Data JPA
│   │   ├── service/         # Logique métier
│   │   └── BankApplication.java
│   └── pom.xml
├── frontend/
│   ├── src/
│   │   ├── services/api.js  # Client Axios
│   │   ├── App.jsx         # Composant principal (SPA)
│   │   └── styles.css
│   └── package.json
```

3. Technologies Utilisées

3.1 Backend

Technologie	Version	Rôle
Java	21+	Langage de programmation principal
Spring Boot	3.3.5	Framework applicatif (auto-configuration, embedded Tomcat)
Spring Web	—	Exposition d'API REST (@RestController, @RequestMapping)
Spring Data JPA	—	Abstraction de la couche d'accès aux données
H2 Database	—	Base de données relationnelle en mémoire (dev/demo)
Lombok	—	Réduction du code boilerplate (getters, constructeurs)
Maven	—	Gestion des dépendances et build

3.2 Frontend

Technologie	Rôle
React	Bibliothèque UI — composants fonctionnels, hooks (useState, useEffect)
Vite	Bundler de développement rapide (HMR, build optimisé)
Axios	Client HTTP pour communiquer avec le backend REST
Lucide React	Bibliothèque d'icônes SVG modernes
CSS moderne	Styles custom (variables CSS, flexbox, design responsive)

4. Base de Données

4.1 Schéma

La base de données H2 est initialisée au démarrage via le fichier data.sql. Elle comprend quatre tables principales :

Table	Clé primaire	Colonnes principales
client	id (BIGINT)	username (unique), phone
app_user	id (BIGINT)	username (unique), password, role, client_id (FK)
account	id (BIGINT)	account_number (unique), balance, client_id (FK)
transaction	id (BIGINT)	type (CREDIT/DEBIT), amount, date, account_id (FK)

4.2 Relations

- Un Client peut posséder plusieurs Comptes (relation One-to-Many)
- Un Compte peut avoir plusieurs Transactions (relation One-to-Many)
- Un AppUser est lié à un Client (relation One-to-One)

4.3 Données de démonstration

Au démarrage, le fichier data.sql insère les données suivantes :

Utilisateur	Mot de passe	Numéro de compte	Solde initial
toufik	123456	ACC-TOUFIK-001	5 000,00 €
mehdi	123456	ACC-MEHDI-001	2 600,00 €
admin	admin123	ACC-ADMIN-001	10 000,00 €

5. Entités JPA

5.1 Client

L'entité Client représente un client bancaire. Elle contient un identifiant unique, un nom d'utilisateur (unique) et un numéro de téléphone.

```
@Entity
public class Client {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Column(nullable = false, unique = true)
    private String username;
    private String phone;
}
```

5.2 Account (Compte bancaire)

L'entité Account représente un compte bancaire. Elle est liée à un Client via une relation Many-to-One et stocke le numéro de compte (unique) et le solde en BigDecimal pour une précision décimale exacte.

```
@Entity
public class Account {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Column(nullable = false, unique = true)
    private String accountNumber;
    @Column(nullable = false)
    private BigDecimal balance = BigDecimal.ZERO;
    @ManyToOne(optional = false)
    private Client client;
}
```

5.3 Transaction

L'entité Transaction enregistre chaque opération sur un compte. Le type est une énumération (CREDIT ou DEBIT), et la date est automatiquement définie à l'heure courante lors de la création.

```
@Entity
public class Transaction {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Enumerated(EnumType.STRING)
    private TransactionType type; // CREDIT | DEBIT
    private BigDecimal amount;
    private LocalDateTime date = LocalDateTime.now();
    @ManyToOne(optional = false)
    private Account account;
}
```

5.4 AppUser

L'entité AppUser gère l'authentification. Elle stocke le nom d'utilisateur, le mot de passe (en clair — à sécuriser en production) et le rôle (CLIENT ou ADMIN). Elle est reliée à un Client par une relation One-to-One.

6. Endpoints de l'API REST

Le backend expose une API REST sur le port 8080. Voici l'ensemble des endpoints disponibles :

6.1 Authentification — /api/auth

Méthode	URL	Description
POST	/api/auth/login	Connexion utilisateur (retourne AuthResponse)
POST	/api/auth/register	Inscription (crée Client + AppUser)
POST	/api/auth/logout	Déconnexion (HTTP 200)

6.2 Clients — /api/clients

Méthode	URL	Description
GET	/api/clients	Lister tous les clients
POST	/api/clients	Créer un nouveau client
PUT	/api/clients/{id}	Modifier un client existant
DELETE	/api/clients/{id}	Supprimer un client

6.3 Comptes — /api/accounts

Méthode	URL	Description
GET	/api/accounts	Lister tous les comptes
POST	/api/accounts	Créer un compte bancaire
PUT	/api/accounts/{id}	Modifier un compte
DELETE	/api/accounts/{id}	Supprimer un compte

6.4 Transactions — /api/transactions

Méthode	URL	Description
GET	/api/transactions	Lister toutes les transactions
GET	/api/transactions/recent	8 dernières transactions (ordre décroissant)

GET	/api/transactions/account/{id}	Transactions d'un compte donné
-----	--------------------------------	--------------------------------

6.5 Virements — /api/transfers

Le service de virement permet de transférer un montant d'un compte source vers un compte destinataire. Le destinataire peut être identifié par son numéro de compte, son numéro de téléphone ou son nom d'utilisateur.

```
POST /api/transfers
Corps de la requête (JSON) :
{
  "fromAccountId": 1,
  "toAccountNumber": "ACC-MEHDI-001",
  "amount": 100
}
```

Règles de validation appliquées par TransferService :

- Le montant doit être strictement positif
- Le compte source et le compte destinataire doivent être différents
- Le solde du compte source doit être suffisant
- Les deux comptes doivent exister

7. Analyse du Code Source

7.1 TransferService — Logique de virement

Le TransferService est la pièce centrale du système. Il est annoté `@Transactional` afin de garantir l'atomicité de l'opération : si une erreur survient, aucun changement n'est persisté.

La résolution du compte destinataire est flexible : le service tente de retrouver le compte par numéro de compte, puis par téléphone, puis par nom d'utilisateur (insensible à la casse). Chaque virement génère deux entrées Transaction — un DEBIT sur le compte source et un CREDIT sur le compte destinataire.

7.2 AuthService — Gestion de l'authentification

L'AuthService gère la connexion et l'inscription. Lors de l'inscription, il crée simultanément un Client et un AppUser liés, dans une transaction unique (`@Transactional`). La vérification du mot de passe est actuellement réalisée en comparaison directe de chaînes (texte clair).

Point d'amélioration : en production, il est indispensable de hacher les mots de passe avec BCrypt ou Argon2 via Spring Security.

7.3 Configuration CORS

La configuration CORS (CorsConfig) autorise toutes les origines, méthodes et en-têtes afin de faciliter le développement local. En production, cette configuration devrait être restreinte aux domaines autorisés.

7.4 Frontend — App.jsx (SPA React)

Le frontend est une Single Page Application React. La navigation entre les vues (Dashboard, Clients, Comptes, Transactions, Virements) est gérée par un état local React sans routeur externe. Axios est utilisé pour tous les appels API via un fichier de services centralisé (api.js).

8. Points de Sécurité et Améliorations

8.1 Problèmes identifiés

Problème	Recommandation
Mots de passe en clair	Utiliser BCryptPasswordEncoder (Spring Security)
Aucune authentification JWT	Implémenter Spring Security + JWT
CORS ouvert (*)	Restreindre aux origines de confiance
H2 en mémoire	Utiliser PostgreSQL ou MySQL en production
Pas de gestion des erreurs globale	Ajouter @ControllerAdvice + @ExceptionHandler

8.2 Améliorations fonctionnelles suggérées

- Pagination et filtrage des transactions
- Historique détaillé des virements
- Gestion des rôles (admin vs client) avec Spring Security
- Tests unitaires et d'intégration (JUnit 5, Mockito)
- Dockerisation (docker-compose pour backend + base de données)

9. Guide de Démarrage

9.1 Backend

```
cd backend
```



```
.\mvnw.cmd spring-boot:run

# Serveur disponible sur : http://localhost:8080
# Console H2              : http://localhost:8080/h2-console
# JDBC URL                : jdbc:h2:mem:bankdb
# Utilisateur             : sa | Mot de passe : (vide)
```

9.2 Frontend

```
cd frontend
npm.cmd install
npm.cmd run dev

# Interface disponible sur : http://localhost:5173
```

10. Conclusion

BankApp est un projet full-stack fonctionnel qui illustre les bonnes pratiques de développement avec Spring Boot et React. L'architecture en couches est claire et facilement extensible. Le système de virement est robuste grâce à la gestion transactionnelle et à la validation métier intégrée.

Pour un déploiement en production, les priorités seraient : la sécurisation de l'authentification avec Spring Security et JWT, le remplacement de la base H2 par une base persistante (PostgreSQL), et la restriction de la configuration CORS. Ces améliorations permettraient de transformer ce projet de démonstration en application bancaire prête à l'emploi.

